

python opps

9.sep.25

In []:

In [5]:

```
a=5
print(a)
print(type(a))
```

```
5
<class 'int'>
```

In [7]:

```
l=[1,2,3,4,5]
print(l)
print(type(l))
```

```
[1, 2, 3, 4, 5]
<class 'list'>
```

In [9]:

```
def config(): # normal funtion
    print("15","8GB")
config()
```

```
15 8GB
```

In [11]:

```
class laptop(): # now method
    def config():
        print("i8","hp",8)
        print("i12","dell",16)
lap1= laptop()
```

In [13]:

```
class laptop():
    def config():
        print("i8","hp",8)
        print("i12","dell",16)
lap1= laptop()
lap1.config()
# lap1.config()
```

```
i8 hp 8
i12 dell 16
```

In [16]:

```
class laptop():
    def config(self): # self is act as argu
        print("i6","hp",12)
        print("i23","dell",13)
lap1= laptop()
lap1.config()
```

```
i6 hp 12
i23 dell 13
```

In [25]:

```
class laptop():
    def config(self):
        print("i3","dell",12)
        print("i2","dell",14)
```

```
# config()          // it wont print
lap1 = laptop() # object is created for class
# lap1.config(lap1) // typeerror
lap1.config()
```

```
i3 dell 12
i2 dell 14
```

```
In [33]: class laptop():
        def config(self):
            print("i4","hp",12)
            print("i4","dell",14)
lap1 =laptop() # here lap1 act as argu
lap1.config(lap1)
```

```
i4 hp 12
i4 dell 14
```

```
In [35]: class laptop():
        def config(self):
            print("i4","hp",12)
            print("i4","dell",14)
lap1 =laptop()
lap2= laptop()
lap3 = laptop()

lap1.config()
```

```
i4 hp 12
i4 dell 14
```

```
In [41]: class computer():
        def config():
            print("i25","8gb")

a=4.3
print(type(a))

lap1= computer()
print(type(lap1))
```

```
<class 'float'>
<class '__main__.computer'>
```

special method in python

```
In [44]: class laptop():
        def __init__(self): # it dont need a calling method
            print("i6", "dell",23)
            print("i6","hp", 22)

lap1=laptop()
```

```
i6 dell 23
i6 hp 22
```

```
In [58]: class laptop():
        def __init__(self): # it dont need a calling method
            print("i6", "dell",23)
            print("i6","hp", 22)
```

```
lap=laptop
print()
lap1=laptop()
print()
lap2 =laptop()
print()
```

```
i6 dell 23
i6 hp 22
```

```
i6 dell 23
i6 hp 22
```

```
In [57]: class laptop():
        def __init__(self): # it dont need a calling method
            print("i6", "dell",23)
            print("i5","hp", 22)

        lap1=laptop()
        print()
        lap2 =laptop()

        lap3.__init__() # you can call directly
```

```
i6 dell 23
i5 hp 22
```

```
i6 dell 23
i5 hp 22
i6 dell 23
i6 hp 22
```

```
In [61]: class computer :
        def __init__(self):
            print("hello mahesh")
        cm1= computer()
        cm2= computer()
        cm3= computer()
```

```
hello mahesh
hello mahesh
hello mahesh
```

```
In [64]: class computer :
        def __init__(self):
            print("hello mahesh")
        def confi():
            print("i4","dell",13455)
        l1=computer()
        l2= computer()
        computer.conf()
```

```
hello mahesh
hello mahesh
i4 dell 13455
```

```
In [75]: class computer :
        def __init__(self, cpu, pro, ram):
            print("hello mahesh")
        def confi(self):
            print("i4", "dell",12)
```

```
print("i7","asure",34)

lap1= computer('i4',"fp",1)
lap2 = computer("i6","hp",23)

lap1.conf()
lap2.conf()
```

```
hello mahesh
hello mahesh
i4 dell 12
i7 asure 34
i4 dell 12
i7 asure 34
```

```
In [79]: class computer():
        def __init__(self, cpu, ram, price):
            self.cpu= cpu
            self.ram= ram
            self.price= price
        def config(self):
            print("configuartion is", self.cpu, self.ram ,self.price)

computer1 = computer("i5",5,5444)
computer2 = computer("i5",5,54224)

#computer1.config()

computer1.config()
computer2.config()
```

```
configuartion is i5 5 5444
configuartion is i5 5 54224
```

```
In [ ]:
```